

DSA Day 3: Pointers -> Python references

Book: Data Structures Through C in Depth - Ch2 §2.2 · pp 18-27 | Code in Python

Overview

C pointers hold addresses. Python has no pointers: names bind to objects. Mutable objects (list, dict) are changed in place through any name that references them; immutables (int, str, tuple) are not.

Key concepts to master

- Name binding vs C address
- `id()` shows identity
- Mutable vs immutable
- Function args are references to objects
- Rebinding inside a function does NOT affect caller

Python implementation

```
def add_item(lst):  
    lst.append(99)      # visible to caller (mutation)  
def rebind(x):  
    x = x + 1           # NOT visible (rebinding a local)
```

Complexity

Dereference / access $O(1)$.

Common pitfalls

- Expecting an int to change after a function call
- Sharing one list reference and mutating it unexpectedly

Practice

- Show `id()` before/after `append`
- Explain why `append()` shows outside but `x=x+1` does not

